

# Ledger Over History

*How one operator and a swarm of agents kept multi-week projects honest, and what to build in five hours today.*

Michael Graham   one private repository, Dec 2025 → Sep 2026

## For the judge, in one paragraph

The brief says long-horizon agents accumulate observations until their own state stops being reliable. I hit that wall ten months ago. Everything below comes from one repository: a message bus and a memory server I wrote in December, a product the swarm built and then measured to death in July, and a chain of formal-mathematics projects that have run for weeks at a time on the harness's own built-in tools. The answer that survived is simple. The agent's history is disposable. The state that matters lives in dated, owned, falsifiable files that a fresh session can read in two minutes. Nothing on the status board is remembered. It is measured again from disk on every tick.

---

3,760

commits since 2025-12-05, 2,451 of them this September

---

9,121

lines in the current gate's ledger, every heading stamped by the clock

---

68

memories the agent wrote for itself: 40 feedback, 11 project, 17 reference

---

927 MB

of session transcripts, all of it disposable

|                                  |                                    |                                            |                                    |                                     |
|----------------------------------|------------------------------------|--------------------------------------------|------------------------------------|-------------------------------------|
| DEC 2025                         | FEB 2026                           | JUL 31 2026                                | AUG 2026                           | SEP 2026                            |
| Custom bus and five-layer memory | Harness ships agent teams built in | The rule fires: 0 of 25, engineering stops | Gates, ledgers, formal mathematics | Board, memory files, built-in tools |

Ten months, one repository. Two leaderboard problems accepted (Aug 28, Sep 13), one Mathlib pull request merged (Sep 11), two gates closed on measurement. The commit volume arrives after the rule fires, once the record could be trusted.

ACT I · DECEMBER 2025

## Building the plumbing myself

The first commit is titled "Add multi-agent infrastructure with 5-layer memory system." I wrote two MCP servers. One wrapped a NATS JetStream bus with six channels so twelve agent roles in tmux panes could claim work, report progress and post blockers. The other was a memory server with five retention tiers, my first explicit answer to the brief's question of what persists and what can be discarded:

| LAYER       | KEPT FOR | HOLDS                                                        |
|-------------|----------|--------------------------------------------------------------|
| Core        | forever  | role, relationships, formative failures that shape behaviour |
| Long-term   | forever  | major insights, milestones                                   |
| Medium-term | 7 days   | patterns noticed this week                                   |
| Recent      | 24 hours | tasks, learnings, conversations                              |
| Compost     | 30 days  | discarded ideas that might come back                         |

A nightly job promoted what mattered and cleared the rest. Agents called `recall_context` on spawn and got a summary, never a transcript. The swarm shipped a real product, CodeRocket, an AI CI/CD generator live on AWS with a marketplace plugin and an npm package. Two things went wrong slowly: the memory files churned as commits nobody read, and as models improved the

project manager started writing code itself, bypassing the specialist roles. The deeper failure was not in the plumbing at all.

ACT II • JULY 29 TO 31, 2026

## The record lied, and a rule fixed it

A verified audit found four things labelled "shipped" that no user could reach. The marketplace pin sat three releases behind. The CI workflow had never run, because it lived in a subdirectory GitHub does not read. "1,541 tests" could not be reproduced from the tree. "Zero users" had been asserted by three consecutive assessments; the database held two.

This was not context overflow. It was the long-horizon failure in its purest form: **state that cannot be checked accumulates as if it were true**. Nothing in the record was dated, owned or falsifiable, so no observation could ever change the plan.

The fix was one governing rule with a date, a threshold, a measurable test, an owner and a kill criterion: twenty-five blind comparisons against a bare coding agent, thirteen wins to survive. The product won zero. Engineering stopped that afternoon; the infrastructure came down two days later. Three rules went into the standing instructions and are still there: **"shipped" means a user can get it; never cite a number you have not run; one snapshot of a thing not happening is not evidence that someone decided against it.**

ACT III • AUGUST TO SEPTEMBER 2026

## Gates as the unit of horizon

The same instrument was then pointed at every new idea before any code was written. Each candidate carries a table of hinges: decidable tests, one day each, with a pass condition and what a failure implies, written down before the work starts. Two rows from a gate opened on September 17:

| HINGE | TEST                                                             | PASS                           | FAIL MEANS                             |
|-------|------------------------------------------------------------------|--------------------------------|----------------------------------------|
| H1    | Re-index three modules from two dimensions to three              | < 20 % rewritten, builds green | kill                                   |
| H2    | One real cohomology cell, kernel-checked, timed on one real cell | ≤ 10 min, ≤ 3.5 GB             | 73 cells over budget: kill or redesign |

The kill rule is pre-registered: any hinge fails, the gate closes, the measurement is recorded, nobody re-prices it by argument. The chain moved to a formal-mathematics leaderboard, where every claim is checked by the Lean 4 kernel. That is the ideal long-horizon testbed, because the machine cannot be talked into agreeing. Results so far: two problems accepted, the second a 176-module, 42,000-line proof built in one day by five workers under one coordinator; one Mathlib pull request merged; two gates closed on measurement; one open gate paused with a resume note.

#### ACT IV · SEPTEMBER 2026

## Moving onto the built-in tools

By February the harness shipped agent teams as a free built-in, and the plumbing stopped needing to be mine. What transferred was not the code but what the custom stack had taught me about state.

**Memory.** The five-tier server gave way to the harness's memory directory, and the tiers became three kinds of file: feedback (a correction and why), project (a constraint not derivable from the code) and reference (a pointer). One fact per file, with a "Why" and a "How to apply". A one-line-per-file index loads at every session start; the files load only when relevant. The agent writes them itself after a correction. The newest one, verbatim:

name: read-the-clock-dont-estimate

On 2026-09-18 I stamped a run of STATUS.md entries "07:50" ...  
"10:30 PDT"

from my own sense of how long the work had taken. The real span was

about 07:41-08:12; the board PNG filenames showed it.

How to apply: every ledger heading gets its time from `$(date "+%H:%M")`

inside the same command. Never type a clock time.

**Bus.** NATS channels and tmux nudges became the harness's subagent and messaging tools. The protocol rules moved with them, each learned on a dated day: a message to an idle agent is a task, so facts go in the ledger and the next brief; inboxes drain oldest-first, so one consolidated message per report; a route change needs an acknowledgement; a module wired into the build is frozen. The concurrency cap was cut from eight to two on measured cost.

**Hooks and ticks.** A pre-tool hook hard-blocks writes to the human-owned site in every permission mode. A status-line hook writes usage figures to a JSON file every sixty seconds, and the board reads that file. A scheduled tick every thirty minutes renders the board to my phone. The session scratchpad is where anything I would regret losing goes to die; the board itself died there once and now lives in the repository.

## The three files that carry a project across days

**The ledger.** Each gate has a `STATUS.md`, append-only, every heading stamped from the clock in the same command that writes it. When I say "find a good place to pause everything," the agent writes a `||` entry naming the gate to resume, the subagent cap, the board tick to re-create, the rulings not yet acted on and the one log to read first. A session started a week later with an empty context is productive in minutes. Beside it sits an obligations ledger, every unproved claim with a column asking whether its hypothesis can be met at all:

An obligation nobody can satisfy is worse than a gap, because it reads as closed.

research/gate-l12/obligations.md, 2026-09-18

**The board.** A 401-line script renders a phone-width image where every row is derived at render time: usage from the hook's JSON, each worker's headline from the last heading in its own report, build bars from measured line counts, machine load from the OS. The only hand-edited input is a small milestone config, and a milestone flips only after the coordinator's own build passes. The only typed text per tick is a one-sentence note.

**The instructions and the memories.** CLAUDE.md carries the incident record with dates; the memory directory carries the corrections. The agent's working context is edited by the agent, reviewed by me, and versioned in git. Transcripts are never read back.

**PERSISTS · IN GIT**

- CLAUDE.md incident record
- STATUS.md ledger per gate
- obligations.md
- roadmap and struck archive
- memory directory, 68 files
- board milestone config
- gate specs with hinges
- hooks in settings.json

*dated, owned, falsifiable*

**DERIVED · EVERY TICK**

- board image to the phone
- usage JSON from the hook
- worker headline and detail
- build bars from line counts
- maintainer replies from GitHub
- machine load
- memory index at spawn

*measured, never remembered*

**DISCARDED · PER SESSION**

- transcripts, 927 MB
- subagent contexts
- scratchpad files
- tmux scrollbar
- NATS stream contents
- the coordinator's sense of elapsed time

*history, the thing that rots*

The split the brief asks for. Anything on the right that turns out to matter is promoted to the left as a file with a date.

| THE BRIEF ASKS FOR                            | WHAT THIS PROJECT DOES                                                                                                              |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Explicit mutable state, not growing histories | Ledgers and a milestone config, appended and flipped under rules; transcripts never read back                                       |
| Agents that edit their own working context    | Feedback memories written after a correction; pause entries written for the next session; instructions amended with dated incidents |
| A split between persist and discard           | Five tiers in December, three memory kinds plus a scratchpad now; the board re-derives the rest                                     |
| Reliability from minutes to days              | Gates with pre-registered kill criteria bound each horizon; a project pauses and resumes from one file                              |

## What to build today

Kickoff is at 11:00; submission opens at 11:30 and closes at 16:30. It needs a public GitHub repository, a short demo video with a shareable link, a description naming the tools used, and the team's names and emails. The project below is a slice of the model-host-agnostic harness I plan to build anyway, small enough for five hours, and backed by a pattern that has already run for weeks.

### THE PROJECT

#### ledger: resumable agent state that survives a change of host

- **One small CLI, five verbs.** `note` appends an entry stamped by the clock. `gate` registers a threshold, date, owner and kill criterion. `obligation` records an unproved claim and whether its hypothesis is satisfiable. `pause` writes the resume entry. `resume` prints the two-minute brief a fresh session needs.

- **Storage is plain Markdown in the repo**, so any host can read it: Claude Code, OpenCode, Codex, or a bare loop against a Bedrock model. No server, nothing to keep alive.
- **The demo.** Start a task under one host, kill it mid-task, resume on a different model on AWS Bedrock from the same ledger, and finish. Judges see explicit mutable state and host independence in one shot.
- **Tinybird is the money.** Best use of Tinybird pays \$2,000, \$1,000 and \$500 in Amazon gift cards, judged by Tinybird's own people in the room. Every ledger append and every usage sample is also an event in Tinybird, so the board becomes one query, and "what changed in the last hour" is SQL a judge can run. This has to be central, not bolted on: the board reads from Tinybird, not from disk.
- **Nimble if the resume demo works by 14:00.** Best use of Nimble pays \$1,000 cash plus credits. A `watch` verb that pulls a topic through Nimble every tick, keeps only dated deltas in the ledger and discards the raw pages is the brief's own diagram running live. Skip it otherwise.
- **Leave the rest.** Liquid AI's prize is a kit plus \$250 and Black Forest Labs pays in credits. A Liquid model on the per-tick distillation step is a real efficiency argument if a teammate has spare time; a FLUX image is decoration.

## The job angle

- **Bring the record, not just the demo.** The 9,121-line ledger, the pause entry, the board images and the accepted solves are multi-week evidence. A demo proves five hours; the record proves ten months.

- **Lead with the July failure.** "Nothing in the record was dated, owned or falsifiable" is a sentence a hiring manager can repeat to a colleague. Then ask the judges from OpenAI, Airbyte, LinkedIn and Induction Labs what their long-running agents fail on, and show the matching memory file.
- **Ship the repo public before 16:30** with a README written in the ledger's own format, and say plainly that the custom bus was overbuilt and the built-ins were enough by February.

---

The history is not the agent. The ledger is. Everything else can be thrown away and measured again tomorrow.